MLOps: Experiment Tracking, Model Registry, and CI/CD for ML

Scaler School of Technology | ML System Design and LLMOps | Week 3 — Session 2/2 | Post-Lecture Review & Practice

Key Takeaways

Track everything from day one

Without experiment tracking you cannot reproduce results, compare runs, or explain why model B beat model A. Track loss curves, all metrics (train/val/test), hyperparameters, data version (git hash or DVC tag), code version, and all artefacts. "Did we try dropout?" should never be an open question.

A model registry is not a file store

S3 gives you `model_v3_final_FINAL.pkl`. A registry gives you semantic versioning, full lineage (what data, code, and hyperparameters produced this model), lifecycle stages (Staging → Production → Archived), and a one-command rollback. The moment you have more than two models in production, you need a registry.

CI/CD for ML adds data and model tests

Standard CI tests code. ML CI tests code, data, and model performance. A new model is only promoted when it passes: schema validation, distribution checks, and a performance gate (new model must beat production on a held-out eval set). Without these gates, you are manually babysitting every deployment.

Distribution shift is mostly catching your own bugs

A monitoring company CTO estimated that 80% of apparent drift on dashboards is caused by internal errors: bugs in data pipelines, missing values incorrectly inputted, wrong feature statistics used for scaling, or inconsistencies between training and serving feature computation. Fix the pipeline before blaming the world.

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

Q1. You run 50 experiments to find the best model. Three months later, your manager asks you to reproduce experiment 27. What information do you need, and what does MLflow track to make this possible?

Q2. Your company stores trained models in S3 with filenames like `fraud_model_v2_latest.pkl`. Name three concrete problems this causes in a team of 6 data scientists, and explain how a model registry solves each.

Q3. A fraud detection model was deployed 6 weeks ago. CTR on flagged transactions has dropped 12% week-over-week but latency is unchanged. List the steps you would take to diagnose whether this is (a) a data pipeline bug, (b) a distribution shift, or (c) a model degradation.

Q4. What are the three types of distribution shift? Give a concrete real-world example of each that is *different* from the COVID search example.

Coding / Design Exercises

Work through these independently or in pairs.

Exercise 1: Design a Model Evaluation Gate

You are building CI/CD for a recommendation system. Design the evaluation gate that decides whether a newly trained model is promoted from Staging to Production. Specify: (a) what dataset you evaluate on and why; (b) what metrics you compute; (c) the promotion rule (e.g. "new model must beat production by X% on metric Y"); (d) what happens if the gate fails. Consider both offline metrics (NDCG@20) and the risk of overfitting to your evaluation set.

Exercise 2: Diagnose a Degraded Model

A sentiment analysis model was deployed 3 months ago on product reviews. Its positive/negative accuracy on your weekly manual eval has dropped from 91% to 78%. Walk through your debugging checklist: What do you check first? What data do you pull? How do you distinguish a data pipeline bug from concept drift? At what point do you decide to retrain vs. roll back?

Exercise 3: Instrument an Existing Pipeline

Take any ML pipeline you have written (classification, regression, anything). Add the following using MLflow: (1) log all hyperparameters at the start of training; (2) log train/val loss at every epoch; (3) log final test metrics; (4) log the trained model as a registered artefact; (5) tag the run with the git commit hash. Run two experiments with different hyperparameters and compare them in the MLflow UI.

Challenge Problem

Full MLOps Pipeline Design

Design a production MLOps pipeline for a ride-sharing demand forecasting system (predicts hourly demand per zone, retrained weekly). Your design must specify: (1) what you track per experiment (be specific — what data version identifier? what metrics?); (2) the model registry lifecycle and promotion criteria; (3) the CI/CD pipeline from code commit to production, including all test stages; (4) what you monitor in production and at what cadence; (5) the three retraining triggers (scheduled, drift-triggered, performance-triggered) with specific thresholds; (6) what constitutes a rollback event and how it is executed. Produce a diagram connecting all components.

Further Reading

- **Chip Huyen — Designing ML Systems, Ch. 6 & 8** — O'Reilly, 2022
  Chapter 6 covers experiment tracking, reproducibility, and offline evaluation. Chapter 8 covers data distribution shifts and monitoring in depth. Both are directly relevant to today's session.
- **MLflow Documentation** — mlflow.org/docs
  The official MLflow docs. Focus on: Tracking, Model Registry, and Projects. Quickstart takes 20 minutes.
- **Evidently AI — ML Monitoring Guide** — evidentlyai.com/blog
  Practical guide to detecting data drift and model degradation in production. Open-source library with excellent documentation.
- **Martin Fowler — Continuous Delivery for ML** — martinfowler.com/articles/cd4ml.html
  The canonical reference for CI/CD applied to ML systems. Explains the deployment pipeline, model evaluation gates, and feature pipelines.

Answers

A1. To reproduce experiment 27 you need: the exact dataset (version/snapshot), the code (git commit hash), all hyperparameters, the random seed, and the environment (Python version, library versions). MLflow tracks all of these when you call `mlflow.log_params()`, `mlflow.log_metrics()`, `mlflow.log_artifact()`, and `mlflow.set_tag("git_commit", ...)`. Without this, experiment 27 is effectively lost.

A2. Three problems with `fraud_model_v2_latest.pkl`: (1) **No lineage** — you can't know what data or code produced it, making debugging impossible; (2) **No lifecycle management** — "latest" means nothing; there's no concept of Staging vs. Production, so you can't safely test a new model before exposing it to users; (3) **No rollback path** — overwriting "latest" means you've lost the previous model and can't quickly revert if the new one is worse. A registry solves all three: lineage is stored with the model, lifecycle stages control promotion, and previous versions are archived and restorable.

A3. Debugging steps: (1) Check the data pipeline first — are features being computed correctly? Are there schema violations, null rate spikes, or distribution changes in the raw data? (2) Compare serving-time feature distributions against training-time distributions (Evidently). If they match, it's not a pipeline bug. (3) Check if the input distribution has shifted (covariate shift) — have transaction patterns changed? New fraud vectors? (4) Check label quality — are the ground truth fraud labels still accurate? (5) If data looks clean but performance has dropped, it's model degradation via concept drift.

A4. (1) **Covariate shift** ($P(X)$ changes): a credit scoring model trained pre-2020 encounters post-COVID income patterns where many previously stable customers have irregular income. (2) **Label shift** ($P(Y)$ changes): a medical diagnosis model trained in one hospital system gets deployed to another where the disease prevalence is different. (3) **Concept drift** ($P(Y|X)$ changes): a housing price model trained in 2019 encounters the 2021-2022 market where the same features (square footage, location) now predict much higher prices due to macroeconomic changes.